

IMDB Sentiment Analysis: Naive Bayes vs LSTM

This notebook:

1. Loads and preprocesses IMDB reviews.
2. Trains a TF-IDF + Naive Bayes model with hyperparameter tuning.
3. Builds and tunes a PyTorch LSTM model.
4. Compares both models using accuracy and confusion matrices.

Important code blocks contain explanatory comments.

Data Loading and Text Preprocessing

```
In [2]: import pandas as pd
import re
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
from bs4 import BeautifulSoup
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Download required resources (run once)
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')

# Load dataset
df = pd.read_csv("IMDB Dataset.csv")

# Initialize tools
lemmatizer = WordNetLemmatizer()

# Keep negation words because they're important for sentiment
stop_words = set(stopwords.words('english'))
stop_words = stop_words - {'not', 'no', 'nor', 'never'}

# Common contractions
contractions = {
    "don't": "do not",
    "doesn't": "does not",
    "didn't": "did not",
    "isn't": "is not",
    "aren't": "are not",
    "wasn't": "was not",
```

```

    "weren't": "were not",
    "can't": "cannot",
    "couldn't": "could not",
    "won't": "will not",
    "wouldn't": "would not",
    "shouldn't": "should not",
    "haven't": "have not",
    "hasn't": "has not",
    "hadn't": "had not",
    "mustn't": "must not",
    "gonna": "going to",
    "wanna": "want to",
    "gotta": "got to",
    "kinda": "kind of",
    "sorta": "sort of"
}

def expand_contractions(text):
    for contraction, expanded in contractions.items():
        text = re.sub(
            r'\b' + re.escape(contraction) + r'\b',
            expanded,
            text,
            flags=re.IGNORECASE
        )
    return text

def preprocess_text(text):

    # Remove HTML tags like <br />
    text = BeautifulSoup(text, "html.parser").get_text(separator=" ")

    # Lowercase
    text = text.lower()

    # Expand contractions
    text = expand_contractions(text)

    # Remove URLs
    text = re.sub(r'http\S+|www\S+|https\S+', '', text)

    # Keep only letters and spaces
    text = re.sub(r'^a-z\s', ' ', text)

    # Remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()

    # Tokenize
    words = text.split()

    # Remove stopwords and lemmatize
    words = [
        lemmatizer.lemmatize(word)
        for word in words
        if word not in stop_words
    ]

    return " ".join(words)

# Apply preprocessing

```

```

df["review"] = df["review"].apply(preprocess_text)

# Encode labels
df["label"] = df["sentiment"].map({
    "negative": 0,
    "positive": 1
})

print(df[["review", "sentiment", "label"]].head())

# save cleaned dataset
df.to_csv("IMDB_Cleaned.csv", index=False)

print("\nDataset shape:", df.shape)
print("\nClass distribution (label):")
print(df["label"].value_counts())
print("\nClass balance (%):")
print((df["label"].value_counts(normalize=True) * 100).round(2))

# -----
# Split data into training and test sets
# -----

from sklearn.model_selection import train_test_split

# X is the review texts, y is the labels (0 or 1)
X = df["review"]
y = df["label"]

# 80% training, 20% test
# stratify=y keeps the same ratio of positive/negative in both sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training reviews:", len(X_train))
print("Test reviews:      ", len(X_test))

```

```

[nltk_data] Downloading package stopwords to
[nltk_data]      C:\Users\b\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]      C:\Users\b\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data]      C:\Users\b\AppData\Roaming\nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!

```

	review	sentiment	label
0	one reviewer mentioned watching oz episode hoo...	positive	1
1	wonderful little production filming technique ...	positive	1
2	thought wonderful way spend time hot summer we...	positive	1
3	basically family little boy jake think zombie ...	negative	0
4	petter mattei love time money visually stunnin...	positive	1

Dataset shape: (50000, 3)

Class distribution (label):

label

1 25000

0 25000

Name: count, dtype: int64

Class balance (%):

label

1 50.0

0 50.0

Name: proportion, dtype: float64

Training reviews: 40000

Test reviews: 10000

Naive Bayes Model and Hyperparameter Tuning

```
In [3]: # NAIVE BAYES
```

LSTM Model, Tuning, and Comparison

```
In [4]: # 3. TF-IDF + Naive Bayes pipeline
pipeline = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("nb", MultinomialNB()),
])

# HYPERPARAMETER TUNING
param_grid = {
    "tfidf__ngram_range": [(1, 1), (1, 2)],
    "tfidf__min_df": [1, 5],
    "tfidf__max_features": [20000, 40000],
    "nb__alpha": [0.1, 0.5, 1.0],
}

grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5, # 5-fold cross-validation
    scoring="accuracy",
    n_jobs=-1, # use all CPU cores
    verbose=1,
)

print("\nRunning GridSearchCV (this can take a few minutes)...")
grid_search.fit(X_train, y_train)

print("\n" + "=" * 50)
print("BEST HYPERPARAMETERS FOUND")
```

```

print("=" * 50)
for param, value in grid_search.best_params_.items():
    print(f" {param:25s} : {value}")
print(f"\nBest cross-validation accuracy: {grid_search.best_score_:.4f}")

# The best model (already refitted on the full training set by GridSearch
best_model = grid_search.best_estimator_

# 4. Evaluate the tuned model on the untouched test set
y_pred = best_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"\n{'=' * 50}")
print(f"TEST ACCURACY (tuned model): {accuracy:.4f}")
print(f"{'=' * 50}\n")

print("Classification Report:")
print(classification_report(
    y_test, y_pred,
    target_names=["negative (0)", "positive (1)"],
))

# 5. Confusion matrix (heatmap)
cm = confusion_matrix(y_test, y_pred)

# Visualize confusion matrix as a heatmap
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["negative", "positive"],
    yticklabels=["negative", "positive"],
    cbar=False,
)
plt.xlabel("Predicted label")
plt.ylabel("True label")
plt.title(f"Naive Bayes (tuned) - Confusion Matrix\nAccuracy: {accuracy:.4f}")
plt.tight_layout() # Save and display figure
plt.savefig("confusion_matrix_nb.png", dpi=150)
print("Saved: confusion_matrix_nb.png")
plt.show()

# Normalized confusion matrix (percentages)
cm_norm = confusion_matrix(y_test, y_pred, normalize="true")
print("\nNormalized confusion matrix (row = true class):")
print(pd.DataFrame(
    cm_norm.round(3),
    index=["true negative", "true positive"],
    columns=["pred negative", "pred positive"],
))

# 6. Most informative words per class
# Identify the strongest positive and negative sentiment words.
# Extract the best TF-IDF vectorizer and Naive Bayes model
best_vectorizer = best_model.named_steps["tfidf"]
best_nb = best_model.named_steps["nb"]

```

```

# Get feature names and calculate sentiment importance scores
feature_names = np.array(best_vectorizer.get_feature_names_out())
log_prob_diff = best_nb.feature_log_prob_[1] - best_nb.feature_log_prob_[0]

# Select top positive and negative words
top_n = 15
top_positive_idx = np.argsort(log_prob_diff)[-top_n:]
top_negative_idx = np.argsort(log_prob_diff)[:top_n]

# Create visualization
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Positive sentiment words
axes[0].barh(
    feature_names[top_positive_idx],
    log_prob_diff[top_positive_idx],
    color="seagreen",
)
axes[0].set_title("Top words -> POSITIVE")
axes[0].set_xlabel("log P(word|pos) - log P(word|neg)")

# Negative sentiment words
axes[1].barh(
    feature_names[top_negative_idx],
    log_prob_diff[top_negative_idx],
    color="indianred",
)
axes[1].set_title("Top words -> NEGATIVE")
axes[1].set_xlabel("log P(word|pos) - log P(word|neg)")

# Save and display figure
plt.tight_layout()
plt.savefig("top_features_nb.png", dpi=150)
print("Saved: top_features_nb.png")
plt.show()

print("\nNaive Bayes finished.")

```

Running GridSearchCV (this can take a few minutes)...
Fitting 5 folds for each of 24 candidates, totalling 120 fits

=====

BEST HYPERPARAMETERS FOUND

=====

```
nb__alpha          : 0.1
tfidf__max_features : 40000
tfidf__min_df       : 1
tfidf__ngram_range  : (1, 2)
```

Best cross-validation accuracy: 0.8804

=====

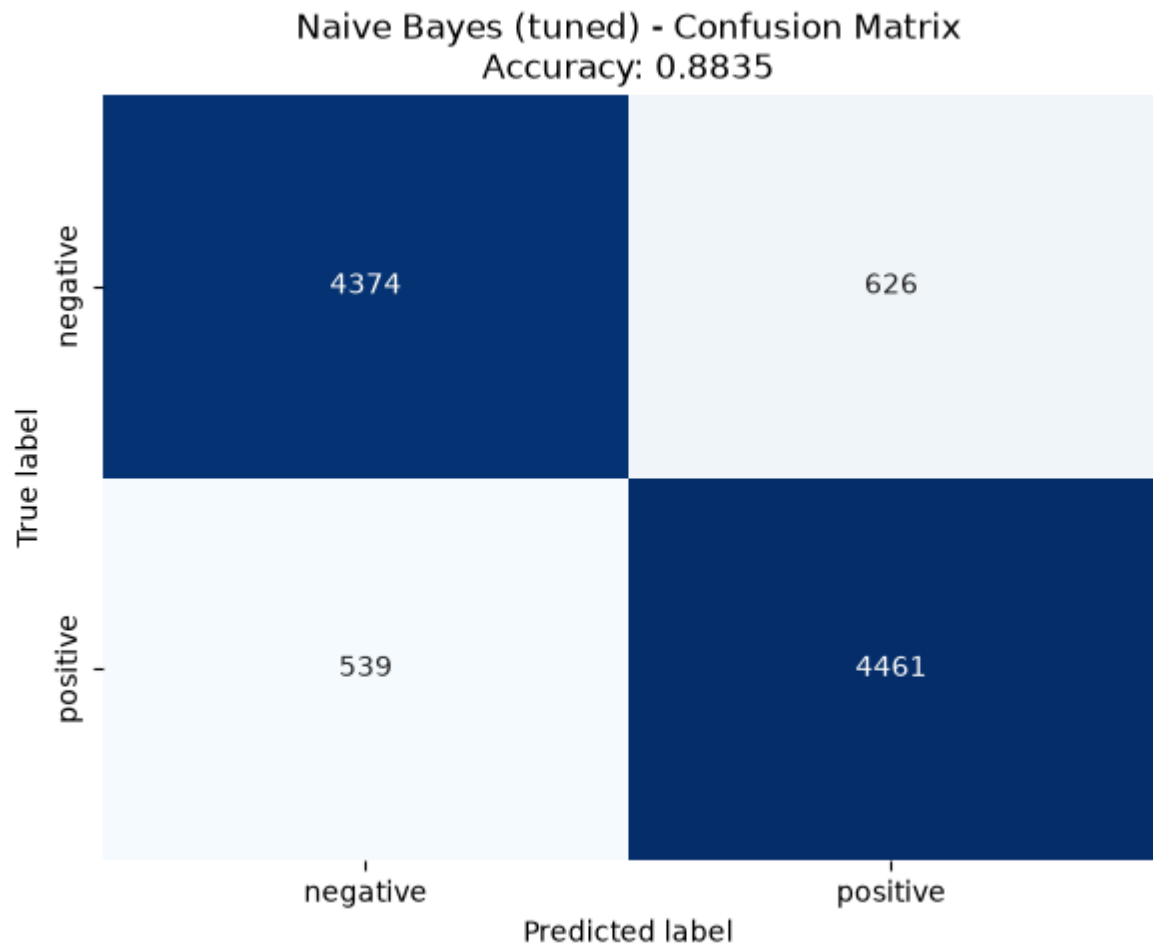
TEST ACCURACY (tuned model): 0.8835

=====

Classification Report:

	precision	recall	f1-score	support
negative (0)	0.89	0.87	0.88	5000
positive (1)	0.88	0.89	0.88	5000
accuracy			0.88	10000
macro avg	0.88	0.88	0.88	10000
weighted avg	0.88	0.88	0.88	10000

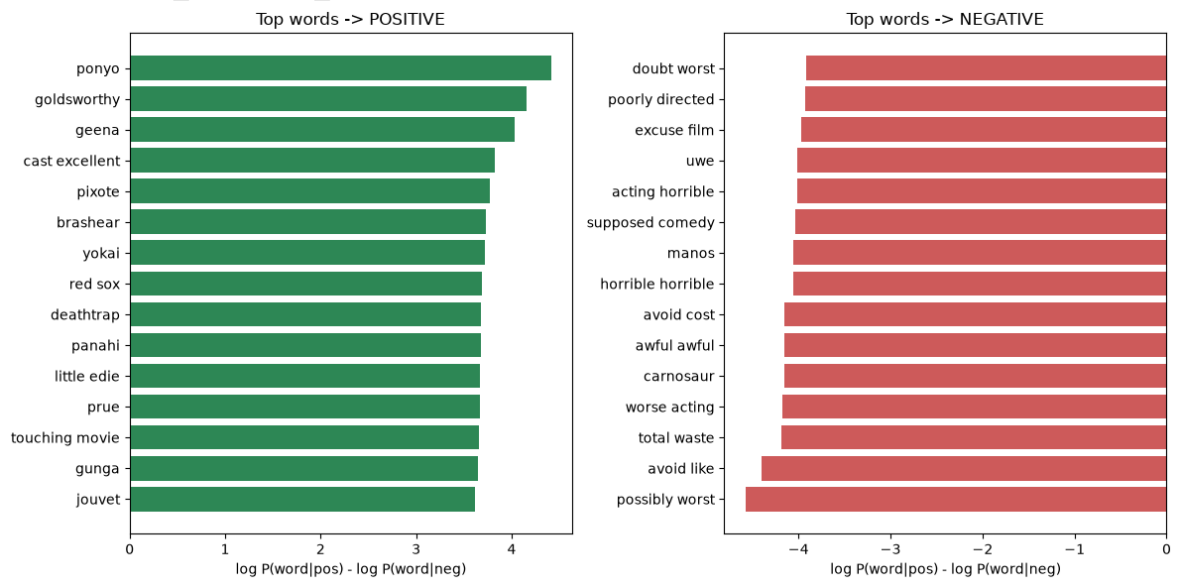
Saved: confusion_matrix_nb.png



Normalized confusion matrix (row = true class):

	pred negative	pred positive
true negative	0.875	0.125
true positive	0.108	0.892

Saved: top_features_nb.png



Naive Bayes finished.

LSTM Model, Tuning, and Comparison

In [5]: `# LSTM`

LSTM Model, Tuning, and Comparison

```
In [6]: import random
import copy
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from collections import Counter

# -----
# LSTM Model using PyTorch
# -----

# ---- Define hyperparameters to try ----

# we list the values we want to try for each parameter
# later we will combine them and test different settings

# we keep the learning rate fixed here because the scheduler will adjust
LEARNING_RATE = 0.001

hyperparameter_options = {
    "vocab_size": [10000, 20000],
    "max_length": [100, 200],
    "embedding_dim": [64, 128],
    "hidden_size": [64, 128],
    "num_layers": [1, 2],
    "batch_size": [32, 64],
```



```

        "dropout": [0.2, 0.5]
    }

    print("Starting LSTM training...")

    # grid search would try all 128 combinations and take hours, so we use random
    # we just pick 5 random combinations and train only those

    NUM_RANDOM_COMBINATIONS = 5

    all_combinations = []

    for vocab_size in hyperparameter_options["vocab_size"]:
        for max_length in hyperparameter_options["max_length"]:
            for embedding_dim in hyperparameter_options["embedding_dim"]:
                for hidden_size in hyperparameter_options["hidden_size"]:
                    for num_layers in hyperparameter_options["num_layers"]:
                        for batch_size in hyperparameter_options["batch_size"]:
                            for dropout in hyperparameter_options["dropout"]:
                                combination = {
                                    "vocab_size": vocab_size,
                                    "max_length": max_length,
                                    "embedding_dim": embedding_dim,
                                    "hidden_size": hidden_size,
                                    "num_layers": num_layers,
                                    "batch_size": batch_size,
                                    "dropout": dropout
                                }
                                all_combinations.append(combination)

    # seed(42) makes the random selection reproducible – we get the same 5 combinations
    random.seed(42)
    selected_combinations = random.sample(all_combinations, NUM_RANDOM_COMBINATIONS)
    print("Total possible combinations:", len(all_combinations))
    print("Combinations we will try:", NUM_RANDOM_COMBINATIONS)

    # ---- Create PyTorch Dataset ----

    class SentimentDataset(Dataset):
        # PyTorch can't work with plain Python lists, it needs a Dataset object
        # so we create our own by extending PyTorch's built-in Dataset class

        def __init__(self, sequences, labels):
            # we convert lists to tensors here
            # sequences use long (integers) because they are word indexes
            # labels use float32 because BCEWithLogitsLoss expects decimal numbers
            self.sequences = torch.tensor(sequences, dtype=torch.long)
            self.labels = torch.tensor(labels, dtype=torch.float32)

        def __len__(self):
            # PyTorch calls this to know how many examples we have
            return len(self.labels)

        def __getitem__(self, index):
            # PyTorch calls this to get one review and its label by position
            one_sequence = self.sequences[index]
            one_label = self.labels[index]
            return one_sequence, one_label

```

```
# ---- LSTM Model ----
```

```
class SentimentLSTM(nn.Module):
    # our model has 3 parts:
    # 1. embedding layer -> each word index becomes a small vector of num
    # 2. LSTM layer -> reads the vectors one by one and keeps a memory
    # 3. output layer -> turns the final memory into a single number (pos

    def __init__(self, vocab_size, embedding_dim, hidden_size, num_layers
        super(SentimentLSTM, self).__init__()

        self.hidden_size = hidden_size

        # each word index is converted to a vector of size embedding_dim
        # padding_idx=0 means the PAD token always gives a zero vector so
        self.embedding = nn.Embedding(
            num_embeddings=vocab_size,
            embedding_dim=embedding_dim,
            padding_idx=0
        )

        # the LSTM reads word vectors in order and updates its memory at
        # batch_first=True just means the input shape is (batch, sequence
        # dropout between layers helps reduce overfitting, but it only wo
        # if we have 1 layer there is nothing between layers, so dropout
        self.lstm = nn.LSTM(
            input_size=embedding_dim,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0.0
        )

        # by default LSTMs forget too much at the start of training, befo
        # setting the forget gate bias to 1 makes it remember more from t
        # this makes training more stable especially in the early epochs
        self._init_forget_gate_bias(num_layers)

        # dropout randomly switches off some neurons during training
        # it stops the model from depending too much on specific neurons
        self.dropout = nn.Dropout(dropout)

        # the output layer takes the final LSTM memory and produces one n
        # sigmoid converts it to a probability between 0 and 1
        # if it's >= 0.5 we say positive, if it's < 0.5 we say negative
        self.output_layer = nn.Linear(hidden_size, 1)

    def _init_forget_gate_bias(self, num_layers):
        # PyTorch packs all 4 gate biases into one tensor in this order:
        # so the forget gate is always at positions [hidden_size : 2 * hi
        # we loop over all layers because each layer has its own set of b
        for layer_idx in range(num_layers):
            for bias_name in [f"bias_ih_l{layer_idx}", f"bias_hh_l{layer_
                bias = getattr(self.lstm, bias_name)
                nn.init.ones_(bias.data[self.hidden_size : 2 * self.hidde

    def forward(self, x):

        # first we turn word indexes into vectors
```

```

        embedded = self.embedding(x)

        # we pass the vectors through the LSTM, it reads them one by one
        # we only care about the final hidden state, not the outputs at e
        lstm_output, (hidden, cell) = self.lstm(embedded)

        # hidden has shape (num_layers, batch, hidden_size), we take the
        last_hidden = hidden[-1]

        # apply dropout before the final prediction
        last_hidden = self.dropout(last_hidden)

        # the output layer takes the ltms's final memory vector and compr
        output = self.output_layer(last_hidden)
        output = torch.sigmoid(output)
        output = output.squeeze(1)

    return output

# ---- Helper functions for the hyperparameter tuning loop ----

def build_vocabulary(X_train, vocab_size):

    # we count how many times each word appears across all training review
    word_counts = Counter()
    for review in X_train:
        words = review.split()
        word_counts.update(words)

    # we take vocab_size - 2 because index 0 and 1 are already taken
    # index 0 = <PAD>, index 1 = <UNK>, so real words start from index 2
    most_common_words = word_counts.most_common(vocab_size - 2)

    # we build a dictionary that maps each word to a number
    # <PAD> and <UNK> are added first, then all the real words follow
    word_to_index = {"<PAD>": 0, "<UNK>": 1}
    for idx, (word, count) in enumerate(most_common_words, start=2):
        word_to_index[word] = idx

    return word_to_index

def convert_to_sequences(X_data, word_to_index, max_length):

    all_sequences = []

    for review in X_data:
        words = review.split()
        sequence = []

        # we replace each word with its index number
        # if the word is not in the vocabulary we use 1 which is <UNK>
        for word in words:
            if word in word_to_index:
                sequence.append(word_to_index[word])
            else:
                sequence.append(1)

        # if the review is too long we keep the end, not the beginning
        # the conclusion and final opinion are usually at the end

```

```

        if len(sequence) > max_length:
            sequence = sequence[len(sequence) - max_length:]

        # we add PAD at the BEGINNING, not the end
        # the LSTM reads left to right and predicts from the last positio
        # so we want the real words to be at the end, not buried under PA
        # example with max_length=5: "great film" -> [0, 0, 0, great_idx,
        while len(sequence) < max_length:
            sequence = [0] + sequence

        all_sequences.append(sequence)

    return all_sequences

def create_dataloaders(X_train_sequences, X_test_sequences, y_train_list,

train_dataset = SentimentDataset(X_train_sequences, y_train_list)
test_dataset = SentimentDataset(X_test_sequences, y_test_list)

# shuffle=True for training so the model doesn't see reviews in the s
# shuffle=False for testing so we always get the same consistent resu
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuff
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle

    return train_loader, test_loader

def train_one_epoch(model, train_loader, criterion, optimizer, device):

    # training mode activates dropout, which is only used during training
    model.train()

    total_loss = 0
    correct = 0
    total_samples = 0

    for batch_sequences, batch_labels in train_loader:

        # data and model must be on the same device (GPU or CPU)
        batch_sequences = batch_sequences.to(device)
        batch_labels = batch_labels.to(device)

        # forward pass: run the batch through the model to get prediction
        predictions = model(batch_sequences)

        # calculate how wrong our predictions are
        loss = criterion(predictions, batch_labels)

        # clear old gradients first, PyTorch adds on top of them by defau
        optimizer.zero_grad()

        # calculate new gradients by going backward through the network
        loss.backward()

        # gradient clipping: if gradients get too large they can break tr
        # this is called the exploding gradient problem, so we cap them a
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        # update the model weights using the gradients we just calculated
        optimizer.step()

```

```

        # Track loss and correct predictions
        total_loss = total_loss + loss.item()
        predicted_labels = (predictions >= 0.5).float()
        correct = correct + (predicted_labels == batch_labels).sum().item()
        total_samples = total_samples + len(batch_labels)

    avg_loss = total_loss / len(train_loader)
    accuracy = correct / total_samples

    return avg_loss, accuracy

def evaluate_model(model, test_loader, criterion, device):

    # evaluation mode turns off dropout so every neuron is active during
    model.eval()

    total_loss = 0
    correct = 0
    total_samples = 0

    # we don't need gradients during evaluation, we're not updating any w
    # no_grad skips all the gradient calculations so it's faster and uses
    with torch.no_grad():

        for batch_sequences, batch_labels in test_loader:

            batch_sequences = batch_sequences.to(device)
            batch_labels = batch_labels.to(device)

            predictions = model(batch_sequences)
            loss = criterion(predictions, batch_labels)

            total_loss = total_loss + loss.item()
            predicted_labels = (predictions >= 0.5).float()
            correct = correct + (predicted_labels == batch_labels).sum().item()
            total_samples = total_samples + len(batch_labels)

    avg_loss = total_loss / len(test_loader)
    accuracy = correct / total_samples

    return avg_loss, accuracy

def run_training(model, train_loader, test_loader, criterion, optimizer,

    NUM_EPOCHS = 50      # we allow up to 50 epochs but early stopping usu
    PATIENCE = 3         # if test accuracy doesn't improve for 3 epochs i
    MIN_DELTA = 0.001    # we only count it as improvement if accuracy goe

    # instead of keeping the same learning rate the whole time, we let th
    # if the test loss doesn't go down for 2 epochs, it cuts the learning
    # so it goes: 0.001 -> 0.0005 -> 0.00025 and so on
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer,
        mode="min",
        factor=0.5,
        patience=2
    )

```

```

best_test_accuracy = 0.0
best_state_dict = None
epochs_without_improvement = 0

for epoch in range(NUM_EPOCHS):
    train_loss, train_accuracy = train_one_epoch(
        model, train_loader, criterion, optimizer, device
    )
    test_loss, test_accuracy = evaluate_model(
        model, test_loader, criterion, device
    )
    print(
        " Epoch", epoch + 1, "/", NUM_EPOCHS,
        " | Train Loss:", round(train_loss, 4),
        " Train Acc:", round(train_accuracy * 100, 2), "%",
        " | Test Loss:", round(test_loss, 4),
        " Test Acc:", round(test_accuracy * 100, 2), "%"
    )
    # we give the scheduler the test loss, not train loss
    # we care about performance on new data, not on the data the model
    # learned on
    scheduler.step(test_loss)
    # we only count it as real improvement if accuracy went up by at
    # least MIN_DELTA
    if test_accuracy > best_test_accuracy + MIN_DELTA:
        best_test_accuracy = test_accuracy
        epochs_without_improvement = 0
        # we use deepcopy to save a snapshot of the weights right now
        # without it, state_dict is just a reference and it would keep
        # changing
        best_state_dict = copy.deepcopy(model.state_dict())
    else:
        epochs_without_improvement = epochs_without_improvement + 1
        if epochs_without_improvement >= PATIENCE:
            print(" Early stopping triggered.")
            break

    return best_test_accuracy, best_state_dict

# ---- Main Hyperparameter Tuning Loop ----

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
y_train_list = y_train.tolist()
y_test_list = y_test.tolist()

all_tuning_results = []
global_best_accuracy = 0.0
global_best_state_dict = None

print("Using device:", device)
print("Starting hyperparameter tuning...")
print()

combination_number = 0

for combination in selected_combinations:

    combination_number = combination_number + 1

    vocab_size = combination["vocab_size"]
    max_length = combination["max_length"]

```

```

embedding_dim = combination["embedding_dim"]
hidden_size = combination["hidden_size"]
num_layers = combination["num_layers"]
batch_size = combination["batch_size"]
dropout = combination["dropout"]

print("-----")
print("Combination      ", combination_number, "/", NUM_RANDOM_COMBINA
print(" vocab_size:      ", vocab_size)
print(" max_length:     ", max_length)
print(" embedding_dim:", embedding_dim)
print(" hidden_size:    ", hidden_size)
print(" num_layers:     ", num_layers)
print(" learning_rate:", LEARNING_RATE, "(fixed)")
print(" batch_size:     ", batch_size)
print(" dropout:        ", dropout)
print()

# Step 1: build vocabulary
word_to_index = build_vocabulary(X_train, vocab_size)

# Step 2: convert reviews to sequences
X_train_sequences = convert_to_sequences(X_train, word_to_index, max_
X_test_sequences = convert_to_sequences(X_test, word_to_index, max_le

# Step 3: create dataloaders
train_loader, test_loader = create_dataloaders(
    X_train_sequences, X_test_sequences,
    y_train_list, y_test_list,
    batch_size
)

# Step 4: create model
model = SentimentLSTM(
    vocab_size=vocab_size,
    embedding_dim=embedding_dim,
    hidden_size=hidden_size,
    num_layers=num_layers,
    dropout=dropout
)
model = model.to(device)

# BCELoss works with probabilities, so we apply sigmoid in the model
criterion = nn.BCELoss()

# Adam is a standard optimizer that works well for most neural network
# it adjusts the learning rate for each weight separately, which make
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)

# Step 6: train the model
best_accuracy, state_dict = run_training(
    model, train_loader, test_loader, criterion, optimizer, device
)

# Save result
result = {
    "combination": combination,
    "accuracy": round(best_accuracy * 100, 2)
}
all_tuning_results.append(result)

```

```

    print(" Best accuracy for this combination:", round(best_accuracy *

# If this combination is the best so far, keep its weights in memory
if best_accuracy > global_best_accuracy:
    global_best_accuracy = best_accuracy
    global_best_state_dict = state_dict

# ---- Find and print the best result ----
best_lstm_result = all_tuning_results[0]
for result in all_tuning_results:
    if result["accuracy"] > best_lstm_result["accuracy"]:
        best_lstm_result = result

print()
print("-----")
print("HYPERPARAMETER TUNING COMPLETE")
print("-----")
print()
print("All LSTM results:")
for result in all_tuning_results:
    print(" Accuracy:", result["accuracy"], "% | Params:", result["com

print()
print("Best LSTM combination:")
for param_name, param_value in best_lstm_result["combination"].items():
    print(" ", param_name, ":", param_value)

print()
print("Best LSTM accuracy:", best_lstm_result["accuracy"], "%")
print("-----")

# -----
# LSTM Confusion Matrix
# -----

# Get the best hyperparameters we found during tuning
best_combo = best_lstm_result["combination"]

# We need to rebuild the vocabulary and sequences using the best hyperpar
# because each combination used different vocab_size and max_length
best_word_to_index = build_vocabulary(X_train, best_combo["vocab_size"])
best_X_test_sequences = convert_to_sequences(X_test, best_word_to_index,

# Create DataLoader only for the test set – we are not training, just eva
best_test_dataset = SentimentDataset(best_X_test_sequences, y_test_list)
best_test_loader = DataLoader(best_test_dataset, batch_size=best_combo["b

# Rebuild the best model and load the best weights from memory
best_lstm_model = SentimentLSTM(
    vocab_size=best_combo["vocab_size"],
    embedding_dim=best_combo["embedding_dim"],
    hidden_size=best_combo["hidden_size"],
    num_layers=best_combo["num_layers"],
    dropout=best_combo["dropout"]
)
best_lstm_model.load_state_dict(global_best_state_dict)
best_lstm_model = best_lstm_model.to(device)

# Switch to evaluation mode (turns off dropout)

```



```

best_lstm_model.eval()

# Go through the test set and collect predictions
lstm_all_preds = []
lstm_all_labels = []

with torch.no_grad():
    for batch_sequences, batch_labels in best_test_loader:

        batch_sequences = batch_sequences.to(device)
        batch_labels = batch_labels.to(device)

        predictions = best_lstm_model(batch_sequences)
        # >= 0.5 means positive, < 0.5 means negative
        predicted_labels = (predictions >= 0.5).float()

        lstm_all_preds.extend(predicted_labels.cpu().tolist())
        lstm_all_labels.extend(batch_labels.cpu().tolist())

# Calculate accuracy and confusion matrix for LSTM
lstm_accuracy = accuracy_score(lstm_all_labels, lstm_all_preds)
lstm_cm = confusion_matrix(lstm_all_labels, lstm_all_preds)

print("LSTM Test Accuracy:", round(lstm_accuracy * 100, 2), "%")

# ---- MODEL COMPARISON PART ----

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Left plot: Naive Bayes
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["negative", "positive"],
    yticklabels=["negative", "positive"],
    cbar=False,
    ax=axes[0]
)
axes[0].set_xlabel("Predicted label")
axes[0].set_ylabel("True label")
axes[0].set_title(f"Naive Bayes\nAccuracy: {accuracy:.4f}")

# right plot: LSTM
sns.heatmap(
    lstm_cm,
    annot=True,
    fmt="d",
    cmap="Oranges",
    xticklabels=["negative", "positive"],
    yticklabels=["negative", "positive"],
    cbar=False,
    ax=axes[1]
)
axes[1].set_xlabel("Predicted label")
axes[1].set_ylabel("True label")
axes[1].set_title(f"LSTM\nAccuracy: {lstm_accuracy:.4f}")

plt.suptitle("Confusion Matrix Comparison", fontsize=14)

```

```

plt.tight_layout()
plt.savefig("confusion_matrix_comparison.png", dpi=150)
print("Saved: confusion_matrix_comparison.png")
plt.show()

# -----
# Accuracy Bar Chart
# -----

model_names = ["Naive Bayes", "LSTM"]
model_accuracies = [accuracy * 100, lstm_accuracy * 100]

plt.figure(figsize=(6, 5))
bars = plt.bar(model_names, model_accuracies, color=["steelblue", "darkorange"])

# Write the accuracy number on top of each bar
for bar, acc in zip(bars, model_accuracies):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.3,
        f"{acc:.2f}%",
        ha="center",
        va="bottom",
        fontsize=11
    )

plt.ylim(0, 110)
plt.ylabel("Test Accuracy (%)")
plt.title("Model Comparison: Naive Bayes vs LSTM")
plt.tight_layout()
plt.savefig("model_comparison.png", dpi=150)
print("Saved: model_comparison.png")
plt.show()

```

Starting LSTM training...
Total possible combinations: 128
Combinations we will try: 5
Using device: cpu
Starting hyperparameter tuning...

Combination 1 / 5
 vocab_size: 10000
 max_length: 100
 embedding_dim: 128
 hidden_size: 128
 num_layers: 2
 learning_rate: 0.001 (fixed)
 batch_size: 32
 dropout: 0.2

Epoch 1 / 50 | Train Loss: 0.4257 Train Acc: 80.1 % | Test Loss: 0.35
57 Test Acc: 84.82 %
Epoch 2 / 50 | Train Loss: 0.27 Train Acc: 89.11 % | Test Loss: 0.296
7 Test Acc: 87.78 %
Epoch 3 / 50 | Train Loss: 0.1949 Train Acc: 92.54 % | Test Loss: 0.3
187 Test Acc: 87.81 %
Epoch 4 / 50 | Train Loss: 0.1267 Train Acc: 95.56 % | Test Loss: 0.3
618 Test Acc: 87.24 %
Epoch 5 / 50 | Train Loss: 0.0761 Train Acc: 97.59 % | Test Loss: 0.4
239 Test Acc: 87.55 %
Early stopping triggered.
Best accuracy for this combination: 87.78 %

Combination 2 / 5
 vocab_size: 10000
 max_length: 100
 embedding_dim: 64
 hidden_size: 64
 num_layers: 2
 learning_rate: 0.001 (fixed)
 batch_size: 64
 dropout: 0.2

Epoch 1 / 50 | Train Loss: 0.501 Train Acc: 74.78 % | Test Loss: 0.39
99 Test Acc: 81.23 %
Epoch 2 / 50 | Train Loss: 0.3316 Train Acc: 85.93 % | Test Loss: 0.3
215 Test Acc: 86.26 %
Epoch 3 / 50 | Train Loss: 0.2646 Train Acc: 89.39 % | Test Loss: 0.3
401 Test Acc: 86.68 %
Epoch 4 / 50 | Train Loss: 0.2171 Train Acc: 91.67 % | Test Loss: 0.3
203 Test Acc: 87.32 %
Epoch 5 / 50 | Train Loss: 0.174 Train Acc: 93.55 % | Test Loss: 0.34
24 Test Acc: 87.15 %
Epoch 6 / 50 | Train Loss: 0.1339 Train Acc: 95.2 % | Test Loss: 0.39
5 Test Acc: 87.27 %
Epoch 7 / 50 | Train Loss: 0.1014 Train Acc: 96.66 % | Test Loss: 0.4
393 Test Acc: 86.94 %
Early stopping triggered.
Best accuracy for this combination: 87.32 %

Combination 3 / 5
 vocab_size: 20000
 max_length: 100

embedding_dim: 64
hidden_size: 64
num_layers: 2
learning_rate: 0.001 (fixed)
batch_size: 64
dropout: 0.2

Epoch 1 / 50 | Train Loss: 0.4949 Train Acc: 75.24 % | Test Loss: 0.3
778 Test Acc: 83.43 %
Epoch 2 / 50 | Train Loss: 0.3255 Train Acc: 86.31 % | Test Loss: 0.3
378 Test Acc: 85.58 %
Epoch 3 / 50 | Train Loss: 0.2525 Train Acc: 89.96 % | Test Loss: 0.3
145 Test Acc: 86.67 %
Epoch 4 / 50 | Train Loss: 0.1972 Train Acc: 92.6 % | Test Loss: 0.34
25 Test Acc: 86.72 %
Epoch 5 / 50 | Train Loss: 0.1486 Train Acc: 94.73 % | Test Loss: 0.3
701 Test Acc: 87.21 %
Epoch 6 / 50 | Train Loss: 0.1067 Train Acc: 96.45 % | Test Loss: 0.4
075 Test Acc: 86.76 %
Epoch 7 / 50 | Train Loss: 0.0609 Train Acc: 98.25 % | Test Loss: 0.5
178 Test Acc: 86.63 %
Epoch 8 / 50 | Train Loss: 0.0441 Train Acc: 98.85 % | Test Loss: 0.5
713 Test Acc: 86.4 %
Early stopping triggered.
Best accuracy for this combination: 87.21 %

Combination 4 / 5
vocab_size: 10000
max_length: 200
embedding_dim: 128
hidden_size: 128
num_layers: 2
learning_rate: 0.001 (fixed)
batch_size: 64
dropout: 0.2

Epoch 1 / 50 | Train Loss: 0.4582 Train Acc: 78.28 % | Test Loss: 0.3
294 Test Acc: 85.87 %
Epoch 2 / 50 | Train Loss: 0.2846 Train Acc: 88.44 % | Test Loss: 0.2
852 Test Acc: 88.14 %
Epoch 3 / 50 | Train Loss: 0.2164 Train Acc: 91.6 % | Test Loss: 0.28
02 Test Acc: 88.43 %
Epoch 4 / 50 | Train Loss: 0.1615 Train Acc: 94.02 % | Test Loss: 0.2
962 Test Acc: 88.76 %
Epoch 5 / 50 | Train Loss: 0.1138 Train Acc: 96.12 % | Test Loss: 0.3
602 Test Acc: 88.49 %
Epoch 6 / 50 | Train Loss: 0.0719 Train Acc: 97.76 % | Test Loss: 0.4
1 Test Acc: 88.2 %
Epoch 7 / 50 | Train Loss: 0.0327 Train Acc: 99.14 % | Test Loss: 0.5
279 Test Acc: 88.17 %
Early stopping triggered.
Best accuracy for this combination: 88.76 %

Combination 5 / 5
vocab_size: 10000
max_length: 200
embedding_dim: 128
hidden_size: 128
num_layers: 1
learning_rate: 0.001 (fixed)

batch_size: 32
dropout: 0.5

Epoch 1 / 50 | Train Loss: 0.4414 Train Acc: 79.09 % | Test Loss: 0.3388 Test Acc: 86.41 %
Epoch 2 / 50 | Train Loss: 0.2687 Train Acc: 89.38 % | Test Loss: 0.2849 Test Acc: 88.35 %
Epoch 3 / 50 | Train Loss: 0.2018 Train Acc: 92.41 % | Test Loss: 0.2791 Test Acc: 88.88 %
Epoch 4 / 50 | Train Loss: 0.1443 Train Acc: 94.84 % | Test Loss: 0.3354 Test Acc: 88.84 %
Epoch 5 / 50 | Train Loss: 0.1004 Train Acc: 96.55 % | Test Loss: 0.3657 Test Acc: 88.66 %
Epoch 6 / 50 | Train Loss: 0.0646 Train Acc: 97.87 % | Test Loss: 0.406 Test Acc: 88.03 %
Early stopping triggered.
Best accuracy for this combination: 88.88 %

HYPERPARAMETER TUNING COMPLETE

All LSTM results:

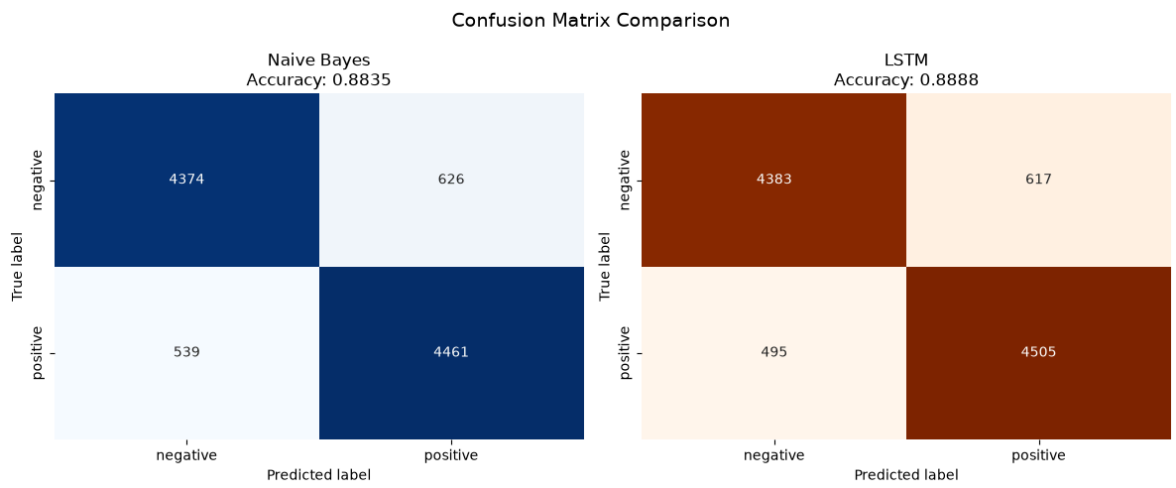
Accuracy: 87.78 % | Params: {'vocab_size': 10000, 'max_length': 100, 'embedding_dim': 128, 'hidden_size': 128, 'num_layers': 2, 'batch_size': 32, 'dropout': 0.2}
Accuracy: 87.32 % | Params: {'vocab_size': 10000, 'max_length': 100, 'embedding_dim': 64, 'hidden_size': 64, 'num_layers': 2, 'batch_size': 64, 'dropout': 0.2}
Accuracy: 87.21 % | Params: {'vocab_size': 20000, 'max_length': 100, 'embedding_dim': 64, 'hidden_size': 64, 'num_layers': 2, 'batch_size': 64, 'dropout': 0.2}
Accuracy: 88.76 % | Params: {'vocab_size': 10000, 'max_length': 200, 'embedding_dim': 128, 'hidden_size': 128, 'num_layers': 2, 'batch_size': 64, 'dropout': 0.2}
Accuracy: 88.88 % | Params: {'vocab_size': 10000, 'max_length': 200, 'embedding_dim': 128, 'hidden_size': 128, 'num_layers': 1, 'batch_size': 32, 'dropout': 0.5}

Best LSTM combination:

vocab_size : 10000
max_length : 200
embedding_dim : 128
hidden_size : 128
num_layers : 1
batch_size : 32
dropout : 0.5

Best LSTM accuracy: 88.88 %

LSTM Test Accuracy: 88.88 %
Saved: confusion_matrix_comparison.png



Saved: model_comparison.png

